案件人物关系管理方案

文档版本：v1.0
编写日期：2026 年 3 月 14 日
开发负责人：王舟

一、需求分析

1.1 典型场景

一个案件往往涉及多个人物，例如：

```
案件：张三诉李四合同纠纷

涉及人物：
├── 当事人
│   ├── 张三（原告）
│   └── 李四（被告）
│
├── 代理人
│   ├── 王五（张三的律师）
│   └── 赵六（李四的律师）
│
├── 证人
│   ├── 钱七（合同签署见证人）
│   └── 孙八（录音提供者）
│
├── 证据链中的人物
│   ├── 周九（收款人，银行流水中出现）
│   ├── 吴十（合同盖章的公司法人代表）
│   └── 郑十一（邮件往来中的联系人）
│
└── 其他相关人员
    ├── 法官
```

```
├── 书记员
└── 鉴定人
```

1.2 核心需求

需求	说明	优先级
人物统一管理	同一人在不同证据中只创建一次	P0
角色分类	区分当事人/证人/代理人/其他	P0
人物 - 案件关联	一个人与多个案件可能有关联	P0
人物 - 证据关联	某人在某证据中出现	P1
人物 - 人物关系	如"张三是李四的代理人"	P1
人物关系图谱	可视化展示人物关系网	P2
人物搜索	按姓名/角色/电话搜索	P1

1.3 人物信息要素

信息类型	字段	说明
基本信息	姓名、性别、身份证号	核心识别信息
联系方式	电话、邮箱、地址	联系用
角色信息	案件角色、与当事人关系	如"原告"、"被告代理人"
扩展信息	工作单位、职务	背景信息
关联信息	关联证据、关联节点、关联人物	关系网

二、数据库设计

2.1 核心表结构

```
-- 人物信息表
CREATE TABLE persons (
    -- 主键
    id                  TEXT PRIMARY KEY,

    -- 基本信息
    name                TEXT NOT NULL,        -- 姓名
    name_pinyin         TEXT,                 -- 姓名拼音（搜索优化）
    gender              TEXT,                 -- 性别: male/female/unknown
    id_number           TEXT,                 -- 身份证号（加密存储）
    id_number_hash      TEXT,                 -- 身份证号哈希（去重）

    -- 联系方式
    phone               TEXT,                 -- 电话
    phone_hash          TEXT,                 -- 电话哈希（去重）
    email               TEXT,                 -- 邮箱
    address             TEXT,                 -- 地址

    -- 扩展信息
    organization        TEXT,                 -- 工作单位
    position            TEXT,                 -- 职务
    notes               TEXT,                 -- 备注

    -- 统计信息（冗余，方便查询）
    related_cases_count INTEGER DEFAULT 0,    -- 关联案件数
    related_evidence_count INTEGER DEFAULT 0, -- 关联证据数
    related_nodes_count INTEGER DEFAULT 0,    -- 关联节点数

    -- 状态
    status              TEXT DEFAULT 'active', -- active/archived/deleted
    created_by          TEXT NOT NULL,
    created_at          DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at          DATETIME DEFAULT CURRENT_TIMESTAMP,

    -- 索引
    INDEX idx_name ON persons(name),
    INDEX idx_name_pinyin ON persons(name_pinyin),
    INDEX idx_phone ON persons(phone),
    INDEX idx_id_hash ON persons(id_number_hash)
);
```

```
-- 人物 - 案件关联表
CREATE TABLE person_case_links (
    id              TEXT PRIMARY KEY,
    person_id       TEXT NOT NULL REFERENCES persons(id),
    case_id         TEXT NOT NULL REFERENCES cases(id),

    -- 角色信息
    role_type       TEXT NOT NULL,        -- 角色类型
    role_detail     TEXT,                 -- 角色详细描述

    -- 时间信息
    involved_date   DATE,                 -- 涉案日期
    created_at      DATETIME DEFAULT CURRENT_TIMESTAMP,

    -- 索引
    INDEX idx_person ON person_case_links(person_id),
    INDEX idx_case ON person_case_links(case_id),
    INDEX idx_role ON person_case_links(role_type),
    UNIQUE INDEX idx_unique (person_id, case_id, role_type)
);

-- 人物 - 证据关联表
CREATE TABLE person_evidence_links (
    id              TEXT PRIMARY KEY,
    person_id       TEXT NOT NULL REFERENCES persons(id),
    evidence_id     TEXT NOT NULL REFERENCES evidence_files(id),

    -- 关联信息
    mention_type    TEXT,                 -- 出现类型: signatory/witness/mentioned
    mention_context TEXT,                 -- 出现上下文（原文片段）
    page_num        INTEGER,              -- 所在页码
    position        INTEGER,              -- 文中位置

    -- 索引
    INDEX idx_person ON person_evidence_links(person_id),
    INDEX idx_evidence ON person_evidence_links(evidence_id)
);

-- 人物 - 节点关联表
CREATE TABLE person_node_links (
    id              TEXT PRIMARY KEY,
    person_id       TEXT NOT NULL REFERENCES persons(id),
    node_id         TEXT NOT NULL REFERENCES event_nodes(id),

    -- 关联信息
    relation_desc   TEXT,                 -- 与该事件的关系描述
```

```
    -- 索引
    INDEX idx_person ON person_node_links(person_id),
    INDEX idx_node ON person_node_links(node_id)
);

-- 人物 - 人物关系表
CREATE TABLE person_person_relations (
    id              TEXT PRIMARY KEY,
    person_a_id     TEXT NOT NULL REFERENCES persons(id),
    person_b_id     TEXT NOT NULL REFERENCES persons(id),

    -- 关系信息
    relation_type   TEXT NOT NULL,        -- 关系类型
    relation_desc   TEXT,                 -- 关系描述
    relation_date   DATE,                 -- 关系发生日期

    -- 关系方向: A→B
    -- 如: 张三是李四的"代理人"

    -- 索引
    INDEX idx_a ON person_person_relations(person_a_id),
    INDEX idx_b ON person_person_relations(person_b_id),
    INDEX idx_relation ON person_person_relations(relation_type),
    UNIQUE INDEX idx_unique (person_a_id, person_b_id, relation_type)
);

-- 角色类型字典表
CREATE TABLE role_types (
    id              TEXT PRIMARY KEY,
    code            TEXT UNIQUE NOT NULL,  -- 角色代码
    name            TEXT NOT NULL,         -- 角色名称
    category        TEXT NOT NULL,         -- 分类: party/agent/witness/other
    description     TEXT,
    color           TEXT                   -- 显示颜色
);
```

2.2 角色类型预设

```
-- 初始化角色类型
INSERT INTO role_types (id, code, name, category, color) VALUES
-- 当事人
('role_001', 'plaintiff', '原告', 'party', '#ef4444'),
('role_002', 'defendant', '被告', 'party', '#3b82f6'),
('role_003', 'third_party', '第三人', 'party', '#f59e0b'),

-- 代理人
```

```
('role_010', 'lawyer', '代理律师', 'agent', '#8b5cf6'),
('role_011', 'legal_rep', '法定代表人', 'agent', '#8b5cf6'),
('role_012', 'agent', '一般代理人', 'agent', '#8b5cf6'),

-- 证人
('role_020', 'witness', '证人', 'witness', '#10b981'),
('role_021', 'expert', '鉴定人', 'witness', '#10b981'),

-- 其他
('role_030', 'signatory', '签署人', 'other', '#6b7280'),
('role_031', 'payee', '收款人', 'other', '#6b7280'),
('role_032', 'contact', '联系人', 'other', '#6b7280'),
('role_033', 'other', '其他', 'other', '#6b7280');
```

2.3 人物关系类型预设

```
-- 初始化人物关系类型
INSERT INTO role_types (id, code, name, category) VALUES
-- 亲属关系
('rel_001', 'spouse', '配偶', 'family'),
('rel_002', 'parent', '父母', 'family'),
('rel_003', 'child', '子女', 'family'),
('rel_004', 'sibling', '兄弟姐妹', 'family'),

-- 工作关系
('rel_010', 'employer', '雇主', 'work'),
('rel_011', 'employee', '雇员', 'work'),
('rel_012', 'colleague', '同事', 'work'),

-- 法律关系
('rel_020', 'agent', '代理人', 'legal'),
('rel_021', 'client', '委托人', 'legal'),
('rel_022', 'guarantor', '担保人', 'legal'),

-- 业务关系
('rel_030', 'partner', '合作伙伴', 'business'),
('rel_031', 'counterparty', '交易对手', 'business');
```

三、Rust 后端实现

3.1 数据模型

```
// src/models/person.rs
use chrono::{NaiveDate, DateTime, Utc};
use serde::{Deserialize, Serialize};
use sqlx::FromRow;
use uuid::Uuid;

/// 人物角色类型
#[derive(Debug, Clone, Serialize, Deserialize, PartialEq)]
#[serde(rename_all = "snake_case")]
pub enum RoleType {
    // 当事人
    Plaintiff,
    Defendant,
    ThirdParty,

    // 代理人
    Lawyer,
    LegalRep,
    Agent,

    // 证人
    Witness,
    Expert,

    // 其他
    Signatory,
    Payee,
    Contact,
    Other,
}

/// 人物信息
#[derive(Debug, Clone, FromRow, Serialize, Deserialize)]
pub struct Person {
    pub id: String,
    pub name: String,
    pub name_pinyin: Option<String>,
    pub gender: Option<String>,
    pub id_number_hash: Option<String>,
    pub phone: Option<String>,
```

```
    pub phone_hash: Option<String>,
    pub email: Option<String>,
    pub address: Option<String>,
    pub organization: Option<String>,
    pub position: Option<String>,
    pub notes: Option<String>,
    pub related_cases_count: i64,
    pub related_evidence_count: i64,
    pub related_nodes_count: i64,
    pub status: String,
    pub created_by: String,
    pub created_at: DateTime<Utc>,
    pub updated_at: DateTime<Utc>,
}

/// 人物 - 案件关联
#[derive(Debug, Clone, FromRow, Serialize, Deserialize)]
pub struct PersonCaseLink {
    pub id: String,
    pub person_id: String,
    pub case_id: String,
    pub role_type: String,
    pub role_detail: Option<String>,
    pub involved_date: Option<NaiveDate>,
    pub created_at: DateTime<Utc>,
}

/// 人物 - 人物关系
#[derive(Debug, Clone, FromRow, Serialize, Deserialize)]
pub struct PersonPersonRelation {
    pub id: String,
    pub person_a_id: String,
    pub person_b_id: String,
    pub relation_type: String,
    pub relation_desc: Option<String>,
    pub relation_date: Option<NaiveDate>,
}

/// 人物完整信息（含关联）
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct PersonWithRelations {
    #[serde(flatten)]
    pub person: Person,
    pub case_links: Vec<PersonCaseLink>,
    pub evidence_links: Vec<PersonEvidenceLink>,
```

```
    pub person_relations: Vec<PersonPersonRelation>,
}
```

3.2 人物服务

```
// src/services/person_service.rs
use sqlx::{SqlitePool, FromRow};
use crate::models::person::*;

pub struct PersonService {
    pool: SqlitePool,
}

impl PersonService {
    pub fn new(pool: SqlitePool) -> Self {
        Self { pool }
   }

   /// 创建人物（自动去重）
    pub async fn create_or_find_person(
        &self,
        input: CreatePersonInput,
    ) -> anyhow::Result<Person> {
       // 1. 尝试通过身份证号哈希查找
        if let Some(id_hash) = &input.id_number_hash {
            if let Some(person) = self.find_by_id_hash(id_hash).await? {
                return Ok(person);
          }
       }

       // 2. 尝试通过电话哈希查找
        if let Some(phone_hash) = &input.phone_hash {
            if let Some(person) = self.find_by_phone_hash(phone_hash).await? {
                return Ok(person);
          }
       }

       // 3. 尝试通过姓名 + 其他信息查找
        if let Some(person) = self.find_by_name_and_info(&input.name, &input).await? {
            return Ok(person);
       }

       // 4. 创建新人物
        let person = Person {
            id: Uuid::new_v4().to_string(),
```

```
            name: input.name,
            name_pinyin: Some(self.convert_to_pinyin(&input.name)),
            gender: input.gender,
            id_number_hash: input.id_number_hash,
            phone: input.phone,
            phone_hash: input.phone_hash,
            email: input.email,
            address: input.address,
            organization: input.organization,
            position: input.position,
            notes: input.notes,
            related_cases_count: 0,
            related_evidence_count: 0,
            related_nodes_count: 0,
            status: "active".to_string(),
            created_by: input.created_by,
            created_at: Utc::now(),
            updated_at: Utc::now(),
        };

        self.insert_person(&person).await?;
        Ok(person)
    }

    /// 关联人物到案件
    pub async fn link_person_to_case(
        &self,
        person_id: &str,
        case_id: &str,
        role_type: &str,
        role_detail: Option<String>,
    ) -> anyhow::Result<PersonCaseLink> {
        // 检查是否已存在
        let existing = self.find_person_case_link(person_id, case_id, role_type).await?;
        if let Some(link) = existing {
            return Ok(link);
        }

        // 创建关联
        let link = PersonCaseLink {
            id: Uuid::new_v4().to_string(),
            person_id: person_id.to_string(),
            case_id: case_id.to_string(),
            role_type: role_type.to_string(),
            role_detail,
            involved_date: None,
```

```
            created_at: Utc::now(),
        };

        self.insert_person_case_link(&link).await?;

        // 更新统计
        self.update_person_stats(person_id).await?;

        Ok(link)
    }

    /// 获取人物完整信息
    pub async fn get_person_with_relations(
        &self,
        person_id: &str,
    ) -> anyhow::Result<PersonWithRelations> {
        let person = self.get_person(person_id).await?;
        let case_links = self.get_person_case_links(person_id).await?;
        let evidence_links = self.get_person_evidence_links(person_id).await?;
        let person_relations = self.get_person_relations(person_id).await?;

        Ok(PersonWithRelations {
            person,
            case_links,
            evidence_links,
            person_relations,
        })
    }

    /// 搜索人物
    pub async fn search_persons(
        &self,
        keyword: &str,
        role_type: Option<&str>,
        case_id: Option<&str>,
        page: i64,
        page_size: i64,
    ) -> anyhow::Result<(Vec<Person>, i64)> {
        // 构建搜索条件
        let mut where_clauses = Vec::new();
        let mut binds = Vec::new();

        if !keyword.is_empty() {
            where_clauses.push("(name LIKE ? OR name_pinyin LIKE ? OR phone LIKE ?)");
            let kw = format!("%{}%", keyword);
            binds.push(kw.clone());
            binds.push(kw.clone());
```

```
            binds.push(kw);
        }

        if let Some(role) = role_type {
            where_clauses.push("person_id IN (SELECT person_id FROM person_case_links WHERE
            binds.push(role.to_string());
        }

        if let Some(cid) = case_id {
            where_clauses.push("person_id IN (SELECT person_id FROM person_case_links WHERE
            binds.push(cid.to_string());
        }

        let where_clause = if where_clauses.is_empty() {
            String::new()
        } else {
            format!("WHERE {}", where_clauses.join(" AND "))
        };

        // 查询总数
        let count_query = format!("SELECT COUNT(*) FROM persons {}", where_clause);
        let total: (i64,) = sqlx::query_as(&count_query).fetch_one(&self.pool).await?;

        // 查询数据
        let data_query = format!(
            r#"
            SELECT * FROM persons
            {}
            ORDER BY created_at DESC
            LIMIT ? OFFSET ?
            "#,
            where_clause
        );

        let mut query = sqlx::query_as::<_, Person>(&data_query);
        for bind in binds {
            query = query.bind(bind);
        }
        query = query.bind(page_size).bind((page - 1) * page_size);

        let persons = query.fetch_all(&self.pool).await?;

        Ok((persons, total.0))
    }

    /// 获取人物关系图谱数据
    pub async fn get_person_graph(
```

```
        &self,
        case_id: &str,
    ) -> anyhow::Result<PersonGraph> {
        // 获取案件中所有人物
        let persons = sqlx::query_as::<_, Person>(
            r#"
            SELECT DISTINCT p.*
            FROM persons p
            JOIN person_case_links pcl ON p.id = pcl.person_id
            WHERE pcl.case_id = ?
          "#,
        )
        .bind(case_id)
        .fetch_all(&self.pool)
        .await?;

        // 获取人物关系
        let relations = sqlx::query_as::<_, PersonPersonRelation>(
            r#"
            SELECT ppr.*
            FROM person_person_relations ppr
            JOIN person_case_links pca ON ppr.person_a_id = pca.person_id
            JOIN person_case_links pcb ON ppr.person_b_id = pcb.person_id
            WHERE pca.case_id = ? OR pcb.case_id = ?
          "#,
        )
        .bind(case_id)
        .bind(case_id)
        .fetch_all(&self.pool)
        .await?;

        Ok(PersonGraph {
            nodes: persons.into_iter().map(|p| GraphNode::from_person(p)).collect(),
            edges: relations.into_iter().map(|r| GraphEdge::from_relation(r)).collect(),
        })
    }
}
```

3.3 人物去重逻辑

```
// src/services/person_dedup.rs

/// 人物去重服务
pub struct PersonDedupService {
    pool: SqlitePool,
```

```
}

impl PersonDedupService {
    /// 查找相似人物（用于去重提示）
     pub async fn find_similar_persons(
         &self,
         input: &CreatePersonInput,
         threshold: f64,
     ) -> anyhow::Result<Vec<(Person, f64)>> {
         let mut candidates = Vec::new();

        // 1. 身份证号匹配（精确）
         if let Some(id_hash) = &input.id_number_hash {
             if let Some(person) = self.find_by_id_hash(id_hash).await? {
                 candidates.push((person, 1.0));
                 return Ok(candidates); // 精确匹配，直接返回
           }
        }

        // 2. 电话匹配（精确）
         if let Some(phone_hash) = &input.phone_hash {
             if let Some(person) = self.find_by_phone_hash(phone_hash).await? {
                 candidates.push((person, 0.95));
           }
        }

        // 3. 姓名 + 其他信息匹配（模糊）
         let similar = self.find_by_similar_name(&input.name, &input).await?;
         for person in similar {
             let score = self.calculate_similarity(&person, input);
             if score >= threshold {
                 candidates.push((person, score));
           }
        }

        // 按相似度排序
         candidates.sort_by(|a, b| b.1.partial_cmp(&a.1).unwrap());

         Ok(candidates)
    }

    /// 计算相似度
     fn calculate_similarity(
         &self,
         existing: &Person,
         input: &CreatePersonInput,
     ) -> f64 {
```

```
        let mut score = 0.0;
        let mut weights = 0.0;

        // 姓名匹配（40% 权重）
        if existing.name == input.name {
            score += 0.4;
        } else if self.is_name_similar(&existing.name, &input.name) {
            score += 0.2;
        }
        weights += 0.4;

        // 电话匹配（30% 权重）
        if let Some(phone) = &input.phone {
            if existing.phone.as_ref() == Some(phone) {
                score += 0.3;
            }
        }
        weights += 0.3;

        // 单位匹配（20% 权重）
        if let Some(org) = &input.organization {
            if existing.organization.as_ref() == Some(org) {
                score += 0.2;
            }
        }
        weights += 0.2;

        // 地址匹配（10% 权重）
        if let Some(addr) = &input.address {
            if existing.address.as_ref() == Some(addr) {
                score += 0.1;
            }
        }
        weights += 0.1;

        score / weights
    }

    /// 合并两个人物
    pub async fn merge_persons(
        &self,
        keep_id: &str,
        merge_id: &str,
    ) -> anyhow::Result<()> {
        let mut tx = self.pool.begin().await?;

        // 1. 更新所有关联到 merge_id 的记录
```

```
        sqlx::query(
            "UPDATE person_case_links SET person_id = ? WHERE person_id = ?"
        )
        .bind(keep_id)
        .bind(merge_id)
        .execute(&mut *tx)
        .await?;

        sqlx::query(
            "UPDATE person_evidence_links SET person_id = ? WHERE person_id = ?"
        )
        .bind(keep_id)
        .bind(merge_id)
        .execute(&mut *tx)
        .await?;

        sqlx::query(
            "UPDATE person_node_links SET person_id = ? WHERE person_id = ?"
        )
        .bind(keep_id)
        .bind(merge_id)
        .execute(&mut *tx)
        .await?;

    // 2. 更新人物关系
        sqlx::query(
            "UPDATE person_person_relations SET person_a_id = ? WHERE person_a_id = ?"
        )
        .bind(keep_id)
        .bind(merge_id)
        .execute(&mut *tx)
        .await?;

        sqlx::query(
            "UPDATE person_person_relations SET person_b_id = ? WHERE person_b_id = ?"
        )
        .bind(keep_id)
        .bind(merge_id)
        .execute(&mut *tx)
        .await?;

    // 3. 标记 merge_id 为合并状态
        sqlx::query(
            "UPDATE persons SET status = 'merged', updated_at = ? WHERE id = ?"
        )
        .bind(Utc::now())
        .bind(merge_id)
```

```
            .execute(&mut *tx)
            .await?;

            tx.commit().await?;

            Ok(())
        }
    }
```

四、API 接口设计

4.1 人物管理接口

方法	路径	说明
POST	/api/v1/persons	创建人物
GET	/api/v1/persons	搜索人物列表
GET	/api/v1/persons/:id	获取人物详情
PUT	/api/v1/persons/:id	更新人物信息
DELETE	/api/v1/persons/:id	删除人物
POST	/api/v1/persons/:id/merge	合并人物

4.2 人物关联接口

方法	路径	说明
POST	/api/v1/persons/:id/cases	关联到案件
DELETE	/api/v1/persons/:id/cases/:case_id	解除案件关联

方法	路径	说明
POST	/api/v1/persons/:id/evidence	关联到证据
POST	/api/v1/persons/:id/relations	添加人物关系

4.3 人物图谱接口

```
GET /api/v1/cases/:case_id/persons/graph
```

响应示例:

```
{
    "code": 200,
    "data": {
        "nodes": [
          {
                "id": "person_001",
                "name": "张三",
                "role": "plaintiff",
                "role_name": "原告",
                "color": "#ef4444",
                "organization": "XXX 公司",
                "position": "总经理"
          },
          {
                "id": "person_002",
                "name": "李四",
                "role": "defendant",
                "role_name": "被告",
                "color": "#3b82f6"
          },
          {
                "id": "person_003",
                "name": "王五",
                "role": "lawyer",
                "role_name": "代理律师",
                "color": "#8b5cf6"
          }
       ],
         "edges": [
          {
```

```
                "source": "person_001",
                "target": "person_003",
                "relation": "agent",
                "relation_name": "代理人",
                "description": "王五是张三的代理律师"
            },
            {
                "source": "person_001",
                "target": "person_002",
                "relation": "counterparty",
                "relation_name": "交易对手",
                "description": "合同签订双方"
            }
        ]
    }
}
```

五、Dioxus 前端实现

5.1 人物列表页面

```
// frontend/src/pages/persons.rs

#[component]
pub fn PersonsPage(case_id: String) -> Element {
    let mut search_keyword = use_signal(|| String::new());
    let mut selected_role = use_signal(|| None);
    let persons = use_resource(move || {
        let kw = search_keyword();
        let role = selected_role();
        async move {
            search_persons(&kw, role.as_deref(), Some(&case_id), 1, 50).await
        }
    });

    rsx! {
        div { class: "persons-page",
            h1 { "案件人物" }

          // 搜索栏
            div { class: "search-bar",
```

```
                input {
                    r#type: "text",
                    placeholder: "搜索姓名/电话/单位...",
                    value: search_keyword(),
                    oninput: move |e| search_keyword.set(e.value()),
                }

                select {
                    oninput: move |e| selected_role.set(Some(e.value())),
                    option { value: "", "全部角色" }
                    option { value: "plaintiff", "原告" }
                    option { value: "defendant", "被告" }
                    option { value: "witness", "证人" }
                    option { value: "lawyer", "律师" }
                }

                button {
                    onclick: move |_| show_add_person_modal(),
                    "添加人物"
                }
            }

            // 人物列表
              match &*persons.value() {
                Some(Ok(data)) => rsx! {
                    div { class: "person-grid",
                        for person in data.persons.iter() {
                            PersonCard {
                                person: person.clone(),
                                onclick: move |_| show_person_detail(&person.id),
                            }
                        }
                    }
                },
                _ => rsx! { div { "加载中..." } }
            }
        }
    }
}

#[component]
fn PersonCard(person: Person) -> Element {
    let role_color = get_role_color(&person.primary_role);

    rsx! {
        div { class: "person-card", style: "border-left: 4px solid {role_color}",
            onclick: move |_| {},
```

```
            div { class: "person-name",
                {person.name}
            }

            if let Some(org) = &person.organization {
                div { class: "person-org", {org} }
            }

            if let Some(pos) = &person.position {
                div { class: "person-position", {pos} }
            }

            div { class: "person-stats",
                span { "📁 {person.related_cases_count} 案件" }
                span { "📄 {person.related_evidence_count} 证据" }
            }
        }
    }
}
```

5.2 人物关系图谱组件

```
// frontend/src/components/PersonGraph.rs

use vis_network::{Network, NetworkData};

#[component]
pub fn PersonGraph(case_id: String) -> Element {
    let graph_data = use_resource(move || fetch_person_graph(&case_id));

    rsx! {
        div { class: "person-graph-container",
            h3 { "人物关系图谱" }

            match &*graph_data.value() {
                Some(Ok(data)) => rsx! {
                    Network {
                        nodes: data.nodes.clone(),
                        edges: data.edges.clone(),
                        options: NetworkOptions {
                            physics: true,
                            hierarchical: false,
                            node_spacing: 150,
                    },
```

```
                        on_node_click: move |node_id| {
                            show_person_detail(node_id)
                        },
                    }
                },
                _ => rsx! { div { "加载图谱..." } }
            }
        }
    }
}
```

5.3 人物详情侧边栏

```
// frontend/src/components/PersonDetail.rs

#[component]
pub fn PersonDetail(person_id: String, on_close: EventHandler<()>) -> Element {
    let person = use_resource(move || fetch_person_with_relations(&person_id));

    rsx! {
        div { class: "person-detail-sidebar",
            button { class: "close-btn", onclick: move |_| on_close.call(()), "×" }

            match &*person.value() {
                Some(Ok(p)) => rsx! {
                    // 基本信息
                    div { class: "section",
                        h3 { "基本信息" }
                        div { class: "info-row", "姓名: {p.person.name}" }
                        if let Some(gender) = &p.person.gender {
                            div { class: "info-row", "性别: {gender}" }
                        }
                        if let Some(phone) = &p.person.phone {
                            div { class: "info-row", "电话: {phone}" }
                        }
                        if let Some(email) = &p.person.email {
                            div { class: "info-row", "邮箱: {email}" }
                        }
                        if let Some(org) = &p.person.organization {
                            div { class: "info-row", "单位: {org}" }
                        }
                        if let Some(pos) = &p.person.position {
                            div { class: "info-row", "职务: {pos}" }
                        }
                    }
```

```
                // 案件关联
                    div { class: "section",
                        h3 { "案件关联" }
                        for link in p.case_links.iter() {
                            div { class: "case-link",
                                span { class: "role-badge", style: "background: {get_role_co
                                    {get_role_name(&link.role_type)}
                                }
                                    span { "案件名称" }
                            }
                        }
                    }

                // 关联证据
                    div { class: "section",
                        h3 { "关联证据 ({p.evidence_links.len()})" }
                        for link in p.evidence_links.iter().take(10) {
                            div { class: "evidence-link",
                                span { "{link.evidence_name}" }
                                if let Some(page) = link.page_num {
                                    span { class: "page-tag", "P{page}" }
                                }
                            }
                        }
                    }

                // 人物关系
                    div { class: "section",
                        h3 { "人物关系 ({p.person_relations.len()})" }
                        for rel in p.person_relations.iter() {
                            div { class: "person-relation",
                                span { "与 " }
                                strong { "{rel.other_person_name}" }
                                span { " 是 " }
                                em { "{rel.relation_desc}" }
                            }
                        }
                    }
                }
            },
                _ => rsx! { div { "加载中..." } }
        }
    }
}
```

六、智能识别与自动关联

6.1 从证据中自动识别人物

```
// src/services/person_extraction.rs

/// 从解析文本中自动识别人物
pub async fn extract_persons_from_evidence(
    evidence: &EvidenceFile,
) -> anyhow::Result<Vec<ExtractedPerson>> {
    let parsed_text = &evidence.parsed_text;

    // 1. 使用 NLP 识别人名
    let names = recognize_person_names(parsed_text);

    // 2. 识别角色上下文
    let mut persons = Vec::new();
    for name_match in names {
        let context = extract_context(parsed_text, &name_match);
        let role = infer_role_from_context(&context);

        persons.push(ExtractedPerson {
            name: name_match.text,
            position: name_match.position,
            context,
            role,
            confidence: name_match.confidence,
       });
    }

    Ok(persons)
}

/// 识别角色
fn infer_role_from_context(context: &str) -> Option<RoleType> {
    if context.contains("原告") || context.contains("上诉人") {
        Some(RoleType::Plaintiff)
    } else if context.contains("被告") || context.contains("被上诉人") {
        Some(RoleType::Defendant)
    } else if context.contains("律师") || context.contains("代理人") {
        Some(RoleType::Lawyer)
    } else if context.contains("证人") {
        Some(RoleType::Witness)
    } else if context.contains("法定代表人") {
```

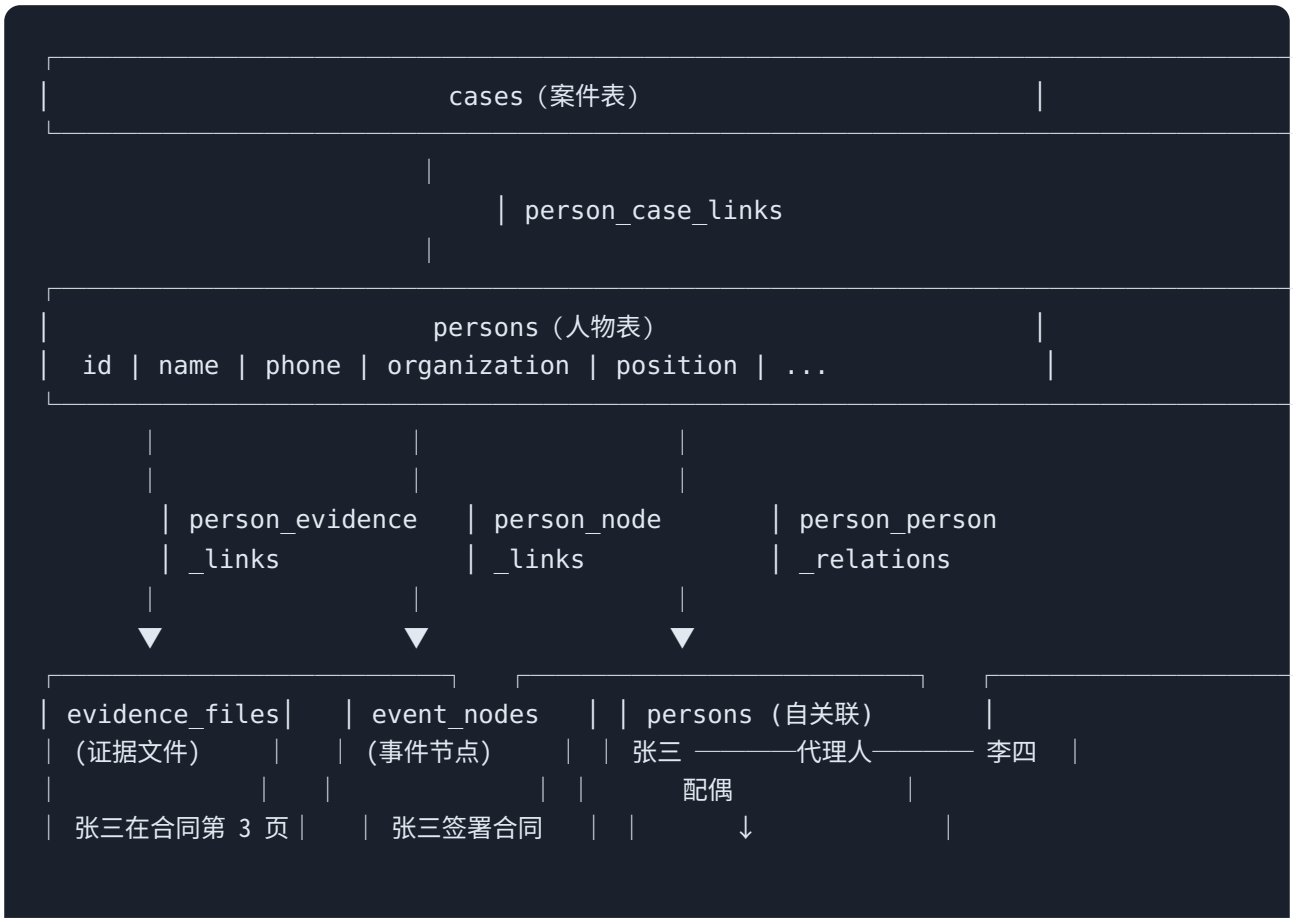
```
        Some(RoleType::LegalRep)
    } else if context.contains("签字") || context.contains("签署") {
        Some(RoleType::Signatory)
    } else {
        Some(RoleType::Other)
    }
}
```

6.2 人物自动关联建议

```
/// 推荐可能的人物关联
pub async fn suggest_person_links(
    person_id: &str,
    case_id: &str,
) -> anyhow::Result<Vec<PersonLinkSuggestion>> {
    let person = get_person(person_id).await?;
    let mut suggestions = Vec::new();

    // 1. 同单位的人
    if let Some(org) = &person.organization {
        let same_org = find_persons_by_organization(org, case_id).await?;
        for p in same_org {
            if p.id != person_id {
                suggestions.push(PersonLinkSuggestion {
                    target_person_id: p.id,
                    relation_type: "colleague",
                    reason: format!("同在{}工作", org),
                    confidence: 0.7,
                });
            }
        }
    }

    // 2. 同电话的人
    if let Some(phone) = &person.phone {
        let same_phone = find_persons_by_phone(phone, case_id).await?;
        for p in same_phone {
            if p.id != person_id {
                suggestions.push(PersonLinkSuggestion {
                    target_person_id: p.id,
                    relation_type: "same_contact",
                    reason: "使用相同联系电话",
                    confidence: 0.9,
                });
            }
```

```
        }
    }

    // 3. 同证据中出现的人
    let co_occurring = find_co_occurring_persons(person_id, case_id).await?;
    for p in co_occurring {
        suggestions.push(PersonLinkSuggestion {
            target_person_id: p.id,
            relation_type: "co_occur",
            reason: format!("在同一证据中出现{}次", p.co_occur_count),
            confidence: 0.5 * p.co_occur_count.min(3) as f64 / 3.0,
        });
    }

    Ok(suggestions)
}
```

七、数据关系图



```
| 签字...          |  |  | 王五 |
```

八、总结

8.1 核心设计

问题	解决方案
同一人在多证据出现	人物表统一管理，通过哈希去重
人物角色多样	person_case_links 表，一人可多角色
人物关系复杂	person_person_relations 自关联表
快速找到某人	姓名拼音索引、电话哈希索引
可视化关系网	图谱数据接口 + vis-network 前端

8.2 性能优化

优化项	说明
拼音索引	支持拼音搜索中文姓名
哈希去重	身份证/电话哈希快速查重
统计冗余	related_count 字段避免实时计算
分页查询	人物列表分页，避免一次加载过多

文档版本：v1.0

最后更新：2026 年 3 月 14 日

开发负责人：王舟

邮箱：main@mails.wedevs.org

电话：15378391447
